

คำแนะนำการส่งโครงงาน

วิชา Computational Geometry 2025

1. วัตถุประสงค์ของโครงงาน

โครงงานนี้มุ่งให้นักศึกษาได้ฝึกปฏิบัติจริงในการ:

- วิเคราะห์ปัญหาเชิงเรขาคณิตคำนวณ (Computational Geometry) อย่างเป็นระบบ
- พัฒนาและเปรียบเทียบอัลกอริทึมระหว่างวิธีพื้นฐาน (Baseline) กับวิธีที่ปรับปรุง (Proposed)
- วัดประสิทธิภาพด้วยตัวชี้วัดเชิงตัวเลข ได้แก่ เวลาทำงาน (runtime), การใช้หน่วยความจำ (memory) และความแม่นยำ (accuracy)
- สร้างผลงานที่มีมาตรฐานระดับงานวิจัยหรือ engineering artifact เพื่อใช้ใน portfolio

2. ภาพรวมของโครงงาน

นิสิตเลือกปัญหาเชิงเรขาคณิตหนึ่งปัญหา เช่น Convex Hull, Voronoi Diagram, Nearest Neighbor Search หรือ Polygon Intersection แล้วดำเนินการตามขั้นตอนดังนี้

- ศึกษาและพัฒนาอัลกอริทึมพื้นฐานที่เรียนในชั้นเรียน
- เสนอแนวทางปรับปรุงด้านเวลา ความแม่นยำ การจัดการข้อมูลขนาดใหญ่ หรือ dynamic update
- ทดสอบและเปรียบเทียบบนชุดข้อมูลทั้งแบบสังเคราะห์ (synthetic) และข้อมูลจริง
- จัดทำรายงานฉบับสมบูรณ์และซอร์สโค้ดที่ทำซ้ำได้ (reproducible)

3. รูปแบบและโครงสร้างของรายงาน

รายงานต้องมีความยาว 6–10 หน้า (ไม่รวมภาคผนวก) เขียนด้วยภาษาไทยหรืออังกฤษ จัดรูปแบบสองคอลัมน์ (IEEE หรือ ACM) ส่งเป็นไฟล์ PDF โดยมีโครงสร้างดังนี้

3.1 หน้าปก

- ชื่อโครงการ
- ชื่อวิชา: Computational Geometry
- ชื่อผู้เขียน และ GitHub username
- ภาคการศึกษา: เทอมปลาย 2568
- คำสำคัญ (keywords) 3–5 คำ

3.2 บทคัดย่อ (Abstract) 150–250 คำ

ต้องระบุส่วนประกอบประกอบ ได้แก่ ปัญหาที่ศึกษา วิธีการที่ใช้ และผลลัพธ์เชิงตัวเลข เช่น “ลดเวลาทำงานลง 42% เมื่อทดสอบกับข้อมูล $n = 100,000$ จุด”

3.3 นิยามปัญหา (Problem Definition)

3.3.1 ที่มาและความสำคัญ

อธิบายปัญหาอย่างคร่าวๆ ว่าต้องการทำอะไร ทำไปเพื่ออะไร

3.3.2 นิยามเชิงรูปนัย (Formal Definition)

องค์ประกอบ	คำอธิบาย
Input	รูปแบบและข้อกำหนดของข้อมูลนำเข้า
Output	ผลลัพธ์ที่คาดหวัง
Constraints	เงื่อนไขบังคับ เช่น จำนวนจุด ความแม่นยำ

3.3.3 ความท้าทาย

- ความซับซ้อนทางเวลา ($O(n \log n)$ เทียบกับ $O(n^2)$)
- ปัญหาความไม่เสถียรเชิงตัวเลข (numerical instability)
- กรณีขอบ (degeneracy cases)

3.4 งานที่เกี่ยวข้องและวิธีพื้นฐาน (Baseline)

สรุปอัลกอริทึมที่เรียนในชั้นเรียนพร้อมตารางเปรียบเทียบข้อดีและข้อจำกัด ดังตัวอย่าง

วิธี (Method)	Time Complexity	ข้อจำกัด (Limitation)
Graham Scan	$O(n \log n)$	ต้องเรียงลำดับก่อน ไม่รองรับ dynamic update
Jarvis March	$O(nh)$	ช้าเมื่อ h มาก เหมาะกับ output-sensitive เท่านั้น

3.5 วิธีการที่เสนอ (Proposed Approach)

- 3.5.1 แนวคิดหลัก (Intuition)
- 3.5.2 การออกแบบอัลกอริทึม (pseudo-code และแผนภาพ)
- 3.5.3 การวิเคราะห์เชิงทฤษฎี: Time Complexity และ Space Complexity

3.6 รายละเอียดการพัฒนา (Implementation Details)

หัวข้อ	รายละเอียด
เทคโนโลยี	ภาษา Python / C++ / Rust และ libraries เช่น NumPy, Shapely, CGAL
โครงสร้างข้อมูล	เช่น KD-tree, Half-edge structure
การจัดการกรณีขอบ	Collinear points และปัญหาความแม่นยำทศนิยม (floating-point precision)

3.7 การทดลอง (Experimental Setup)

หัวข้อ	รายละเอียด
ชุดข้อมูล	ข้อมูลสังเคราะห์ (random, grid, worst-case) และข้อมูลจริงถ้ามี ขนาด $n = 1k, 10k, 100k$
สภาพแวดล้อม	ระบุ CPU, RAM, OS
ตัวชี้วัด	Runtime (ms), Memory usage, Accuracy (สำหรับปัญหา

	approximate), Scalability
--	---------------------------

3.8 ผลการทดลองและการวิเคราะห์ (Results & Evaluation)

Input Size	Baseline (ms)	Proposed (ms)	Speedup
1,000	—	—	—
10,000	—	—	—
100,000	—	—	—

นอกจากตารางแล้ว ต้องมีการแสดงผลเชิงภาพ (กราฟ, ไดอะแกรม หรือ output ทางเรขาคณิต) และอภิปรายผลว่าเหตุใดวิธีที่เสนอจึงดีขึ้นหรือไม่ดีขึ้น รวมถึงกรณี worst-case

3.9 อภิปรายผลโดยรวม, บทสรุป, รายการอ้างอิง และภาคผนวก

- ส่วนที่ 9 Discussion: จุดแข็ง ข้อจำกัด และ trade-off
- ส่วนที่ 10 Conclusion: สิ่งที่ได้ ทำได้ ผลกระทบ และงานต่อยอดในอนาคต
- ส่วนที่ 11 References: ใช้รูปแบบ IEEE หรือ ACM
- ส่วนที่ 12 Appendix (optional): proof, การทดลองเสริม หรือ pseudo-code เติมรูปแบบ

4. โครงสร้างโฟลเดอร์และไฟล์ที่ต้องส่ง

ส่งซอร์สโค้ดแบบ reproducible พร้อม README.md โดยใช้โครงสร้างต่อไปนี้

```
computational-geometry-project/  
├── README.md  
├── report/  
│   └── final_report.pdf  
├── src/  
│   ├── main.py  
│   ├── algorithms/  
│   │   ├── baseline.py  
│   │   └── proposed.py  
│   ├── data_structures/  
│   │   ├── kd_tree.py  
│   │   └── geometry_utils.py  
├── experiments/  
│   ├── benchmark.py  
│   └── results/  
│       ├── runtime.csv  
│       └── plots.png  
└── data/
```

```

├── synthetic/
├── real/
├── visualization/
│   └── plot_results.py
├── demo/
│   ├── README.md
│   └── (ไฟล์ demo ตามรูปแบบที่เลือก - ดูหมวดที่ 8)
├── tests/
│   └── test_algorithms.py
└── requirements.txt (หรือ CMakeLists.txt, Cargo.toml)

```

เนื้อหาขั้นต่ำใน README.md

```

# Project Title

## Problem
คำอธิบายสั้น ๆ ของปัญหาที่เลือก

## Approach
- Baseline: ...
- Proposed: ...

## Results
- Speedup: XX%
- Dataset: ...

## How to Run
pip install -r requirements.txt
python src/main.py

```

5. ข้อกำหนดทางเทคนิคที่สำคัญ

เพื่อให้โครงงานมีคุณภาพระดับ portfolio ต้องมีองค์ประกอบครบทั้ง 3 ข้อดังนี้

ข้อ	ข้อกำหนด	รายละเอียด
1	เปรียบเทียบ Baseline	ห้าม implement อัลกอริทึมเดียว ต้องแสดงให้เห็นว่าวิธีที่เสนอดีกว่าหรือแย่กว่าอย่างไร พร้อมตัวเลขประกอบ
2	Scaling Test	ทดสอบกับข้อมูลอย่างน้อย 3 ขนาด เช่น $n = 1,000$ $10,000$ และ $100,000$ จุด
3	Visualization	ต้องมีรูปประกอบ เช่น Voronoi diagram, convex hull animation หรือกราฟ runtime vs input size

6. เกณฑ์การประเมิน

ในส่วนของ Project Deliverable คิดเป็น 40% ของคะแนนวิชานี้ ประกอบด้วยอัตราส่วนต่อไปนี้

ส่วนประกอบ	คะแนน (%)
ความถูกต้องของอัลกอริทึมและการออกแบบ	25%
การทดลองและการเปรียบเทียบ (Baseline + Scaling)	25%
คุณภาพของรายงาน (รูปแบบ ภาษา ความชัดเจน)	20%
ความสามารถในการทำซ้ำ (Reproducibility) — โค้ดและ README	15%
Demo และความคิดสร้างสรรค์ (interactive, video, animation)	15%

5% มาจาก Project Proposal ที่ส่งไปแล้ว และอีก 5% มาจากการพรีเซนตในวันที่ 6 พ.ค.

⚠ สำคัญ	โครงการที่ไม่มี Baseline หรือ Scaling Test จะได้คะแนนส่วนการทดลองไม่เกิน 50% ของส่วนนั้น
----------------	--

ช่องทางการส่งงาน

อัปโหลดไฟล์ .zip มาในระบบ myCourseVille

8. ข้อกำหนดการส่ง Demo (บังคับ)

การส่ง Demo ถือเป็นข้อบังคับสำหรับโครงการนี้ ไม่ใช่ส่วนเสริม เพื่อให้นิสิตแสดงผลงานได้อย่างเป็นรูปธรรม และให้อาจารย์ตรวจสอบความถูกต้องของอัลกอริทึมได้ง่ายขึ้น

8.1 รูปแบบ Demo ที่ยอมรับ

เลือกส่งได้หนึ่งประเภทจากสามประเภทดังนี้

	ประเภท	คำอธิบาย	จุดเด่น
A	Video Demo	คลิปวิดีโอ 1-2 นาที อัปโหลดที่	เห็นภาพรวมการทำงาน

		YouTube (Unlisted หรือ Public) แบนลิงก์ใน README.md	เหมาะกับอัลกอริทึมที่แสดงผลทางเรขาคณิตได้สวยงาม
B	Interactive Demo	หน้าเว็บหรือ Jupyter Notebook ที่ปรับพารามิเตอร์หรือเลือกข้อมูลได้แบบ real-time	แสดงความยืดหยุ่นของโค้ด เหมาะกับโครงการที่ทดสอบหลายเงื่อนไข
C	CLI Demo Script	สคริปต์ที่รันแล้วแสดง visualization และผลลัพธ์โดยอัตโนมัติ (บันทึกเป็นภาพหรือ pop-up)	ตรวจสอบซ้ำได้ง่าย ไม่ต้องพึ่งพาแพลตฟอร์มออนไลน์

หมายเหตุ (C)	สคริปต์ต้องทำงานได้ทันทีหลังติดตั้ง dependencies ด้วยคำสั่งเดียว เช่น python demo.py หรือ make demo
--------------	---

8.2 เนื้อหาที่ต้องปรากฏใน Demo

Demo ต้องแสดงให้เห็นอย่างน้อย 3 สิ่งต่อไปนี้

1. **อินพุตตัวอย่าง** — ชุดข้อมูลที่ใช้ เช่น จุดแบบสุ่ม รูปหลายเหลี่ยม หรือข้อมูลจากไฟล์
2. **การทำงานของอัลกอริทึม** — ภาพขั้นตอนการคำนวณ เช่น convex hull แบบ incremental หรือการเคลื่อนที่ของ sweep line
3. **ผลลัพธ์** — ผลลัพธ์ทางเรขาคณิต (hull, Voronoi diagram, closest pair) พร้อมตัวเลขประสิทธิภาพ (runtime และจำนวนจุดที่ประมวลผล)

สำหรับประเภท A และ B แนะนำให้ใส่คำบรรยาย (subtitle หรือ voiceover) เพื่อความชัดเจน

8.3 โครงสร้างโฟลเดอร์ demo

demo/	
├─ README.md	คำอธิบายวิธีดูหรือรัน demo
├─ video_link.txt	(ประเภท A) ลิงก์ YouTube
├─ interactive/	(ประเภท B) source code ของ interactive demo

```
├─ app.py หรือ notebook.ipynb
├─ requirements_demo.txt
├─ cli_demo.py (ประเภท C) สคริปต์รัน demo อัตโนมัติ
├─ screenshots/ (optional) ภาพตัวอย่างสำรอง
```

8.4 เกณฑ์การประเมิน Demo

Demo มีน้ำหนักคะแนน 15% ของโครงงานทั้งหมด (คำนวณแทนส่วน Visualization และความคิดสร้างสรรค์)

องค์ประกอบ	น้ำหนักคะแนน (%)
ความชัดเจน — เข้าใจการทำงานได้โดยไม่ต้องอ่านโค้ด	40%
ครบถ้วน — แสดงอินพุต กระบวนการ ผลลัพธ์ และประสิทธิภาพ	30%
คุณภาพทางเทคนิค — ความละเอียดวิดีโอ ความเสถียร interactive การแสดงผลที่สวยงาม	20%
ความคิดสร้างสรรค์ — animation, การเปรียบเทียบแบบ side-by-side	10%

8.5 ข้อควรระวัง

- ห้ามส่ง Demo ที่แสดงแต่ข้อความบน terminal โดยไม่มีภาพทางเรขาคณิต (หากอัลกอริทึมไม่สามารถแสดงภาพได้ ต้องขออนุญาตอาจารย์ล่วงหน้า)
- Video Demo ต้องเน้นเนื้อหา ไม่ใช่การแนะนำด้วยยาวเกินไป
- Interactive Demo แบบเว็บต้องรันบนเครื่องของอาจารย์ได้ หรือ deploy บนบริการฟรี เช่น Streamlit Cloud หรือ Hugging Face Spaces แล้วแจ้งลิงก์

8.6 ตัวอย่าง Demo ที่ดี

หัวข้อโครงงาน	สิ่งที่ควรแสดงใน Demo
Convex Hull	จุดเริ่มต้น → เรียงลำดับ → Graham scan ทีละขั้น → ไฮไลต์ hull สุดท้าย
Voronoi Diagram	เปลี่ยนสี region ตาม seed, แสดงการขยายตัวด้วย Fortune's algorithm animation
Nearest Neighbor Search	slider ปรับตำแหน่ง query point และแสดง nearest point

9. คำแนะนำเพื่อคะแนนสูงและ Portfolio โดดเด่น

- ทำคลิปสาธิต (demo video) 1–2 นาที อัปโหลดเป็น Unlisted บน YouTube แแนบลิงก์ใน README
- สร้างหน้าเว็บ interactive ด้วย Streamlit หรือ Jupyter Notebook แสดงผลแบบ real-time
- เปรียบเทียบกับไลบรารีมาตรฐาน เช่น CGAL, Shapely หรือ SciPy เพื่อแสดง trade-off อย่างรอบด้าน
- ใช้ Git อย่างเป็นระบบ มี commit history ชัดเจน และ branch สำหรับ feature หรือ experiment